VPC & EC2 Web Server Deployment

Subnet design, public/private routing and an HTTP application

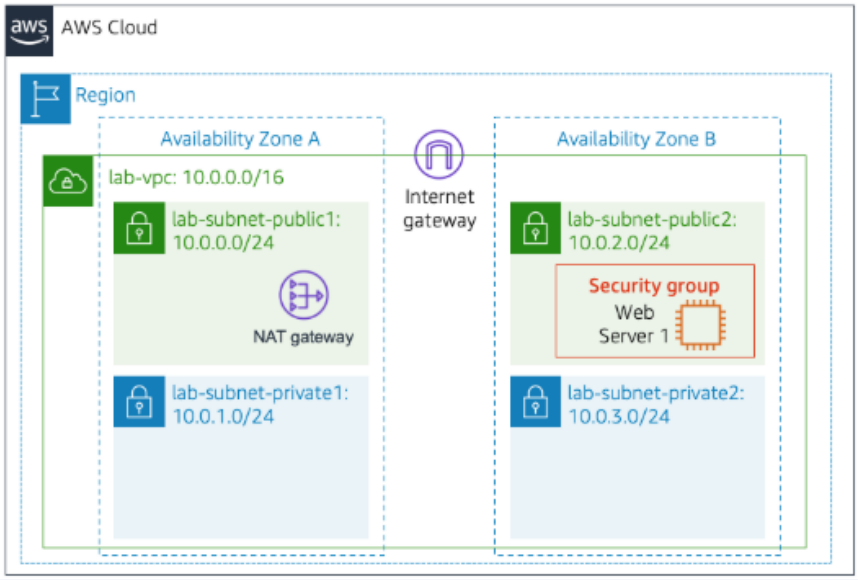
AUTHOR

Younes Chemmaoui

REPORT DATE

7 November 2026

LAB ARCHITECTURE



Public Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Internet gateway

Private Route Tables

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	NAT gateway

Original deployed-lab architecture: two Availability Zones, four subnets, NAT and an EC2 web server.

LAB SCOPE

A custom VPC, four /24 subnets, Internet Gateway and NAT routing, HTTP security group, and EC2 bootstrap.

DOCUMENTED RESULT

30 / 30

Captured lab grade, with the HTTP application demonstrated in the browser.

EVIDENCE & DOCUMENTATION

24 original images | Address plan, routing, deployment and validation

Console captures dated 5-7 November 2026.

https://github.com/YOUNES0101/aws-vpc-web-server-lab

AWS VPC and EC2 Web Server: Detailed Lab Report

Younes Chemmaoui
Prepared: 7 November 2026
Source captures dated 5-7 November 2026

1. Objective, scope and recorded outcome

I built a custom Amazon VPC, extended it with public and private subnets in a second Availability Zone, configured subnet route-table associations, created an HTTP security group, and launched an EC2 web server. The aim was to understand how addressing, routing and instance-level traffic rules combine to make a web application reachable.

Unlike a lab that only inspects preconfigured policies, this exercise involved creating the VPC resources and the server through the AWS Management Console. The training account, permissions, key pair and application download were provided by the lab environment. I do not claim ownership of the supplied application or the pre-existing background resources visible in the account.

The final source captures show the application loaded over HTTP, an EC2 instance in a public subnet, and a grading summary of 30/30. The grader reports six checks at 5/5: VPC creation, new subnets, route-table association, security group, EC2 instance and website accessibility.

Evidence basis

This report uses the main working note and its linked Subnet CIDR blocks note. All 24 local source images are included with their original names and unchanged bytes. The original captures contain annotations, repeated views and conceptual architecture diagrams; none were edited for publication.

The source spans several dates and account/resource identifiers. I use the captures to document the workflow and its results without presenting every image as one continuous session. Where an image shows a transitional state, I describe that state rather than treating it as final configuration.

Method and main finding

I followed the Console workflow from VPC creation through subnet expansion, routing, security-group creation, EC2 launch and browser testing. I compared configuration views with the intended topology and the grading result. No AWS resources or network tests were newly executed while preparing this report.

The main finding is that an address range and a running instance are not enough for public reachability. The subnet route, a public IPv4 address, the security-group rule, the web-service listener and the request protocol must agree. The successful browser page demonstrates that the captured HTTP path worked.

2. Lab architecture

The planned final topology uses lab-vpc (10.0.0.0/16), four /24 subnets across two Availability Zones, an Internet Gateway, one NAT Gateway and a web server in the second public subnet. The original diagrams below show the intended relationships and the public/private route-table distinction.

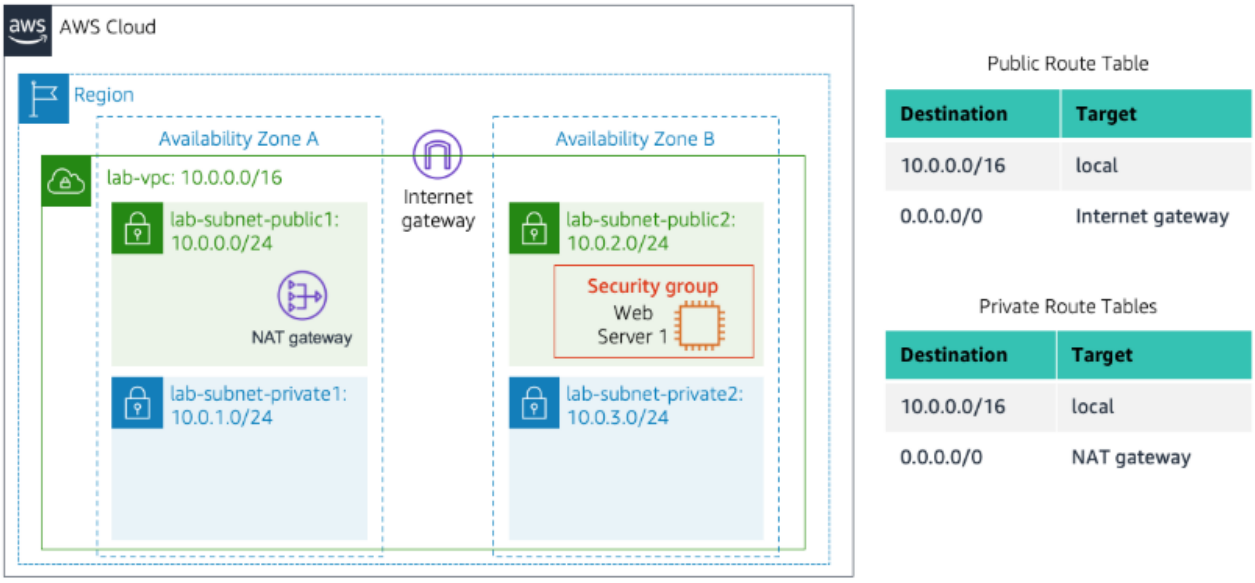


Figure 1. Original scenario diagram: two Availability Zones, four subnets and Web Server 1 in public subnet2.

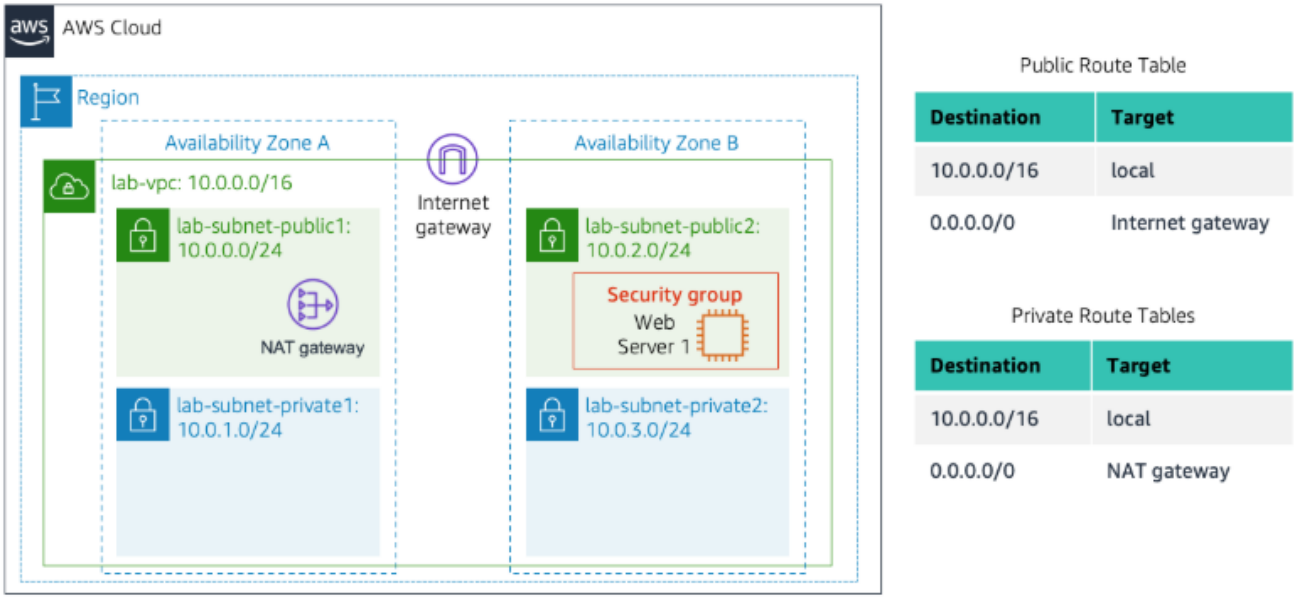


Figure 2. Original completed-architecture diagram retained in the note and embedded near the top of the README.

These diagrams are explanatory models. The actual resource IDs differ across the source sessions. The second public subnet is the server placement used in the exercise; the private subnets are included to study outbound routing through NAT rather than to host a second web tier.

3. Creating the initial VPC resources

I opened the VPC console in us-east-1 and used VPC and more. The note records the lab name prefix, IPv4 range 10.0.0.0/16, one initial Availability Zone, one public subnet, one private subnet, one NAT Gateway, no VPC endpoint and enabled DNS hostnames/resolution.

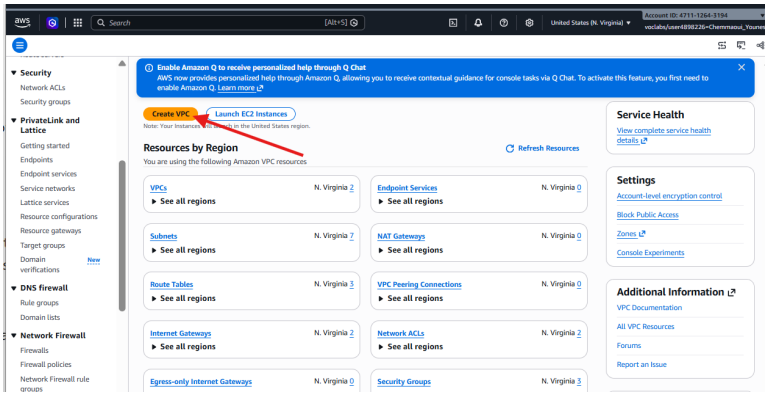


Figure 3. Original VPC dashboard and entry point for Create VPC.

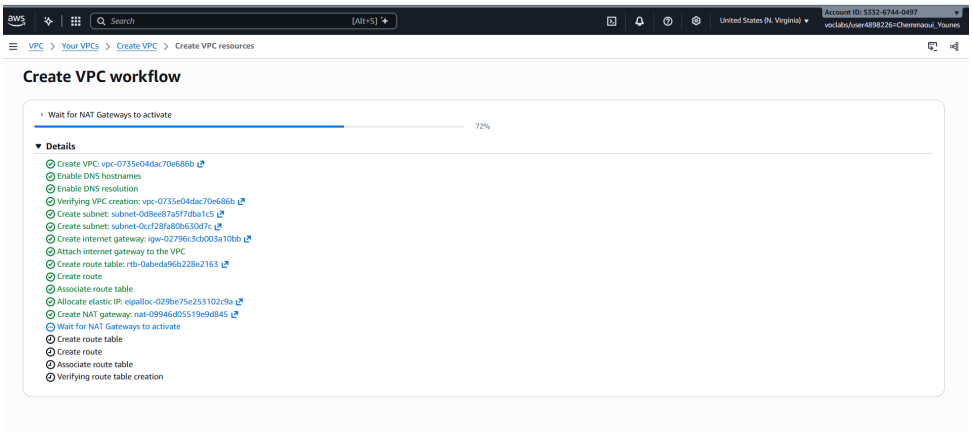


Figure 4. VPC creation workflow at a transitional stage while the NAT Gateway is still activating.

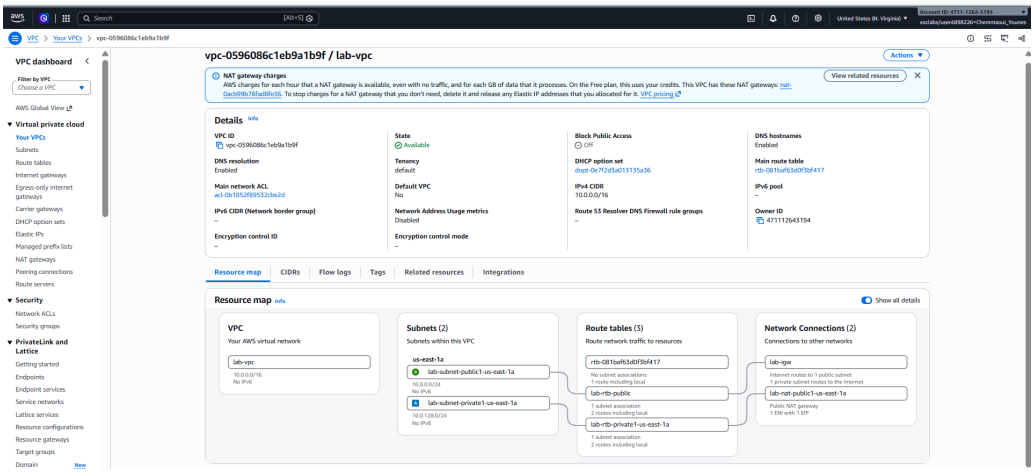


Figure 5. VPC details and resource-map view after the initial provisioning workflow.

The progress image is not presented as a completed NAT status: it explicitly shows a wait step. The later resource views and the final grading record support eventual creation. The VPC wizard created several related resources together, including gateways, subnets and route tables.

4. Initial subnet layout and address plan

The first Availability Zone contains a public subnet at 10.0.0.0/24 and a private subnet at 10.0.1.0/24. The public route table sends internet-bound IPv4 traffic to the Internet Gateway; the private route table sends that traffic to NAT. Both also retain a VPC-local route.

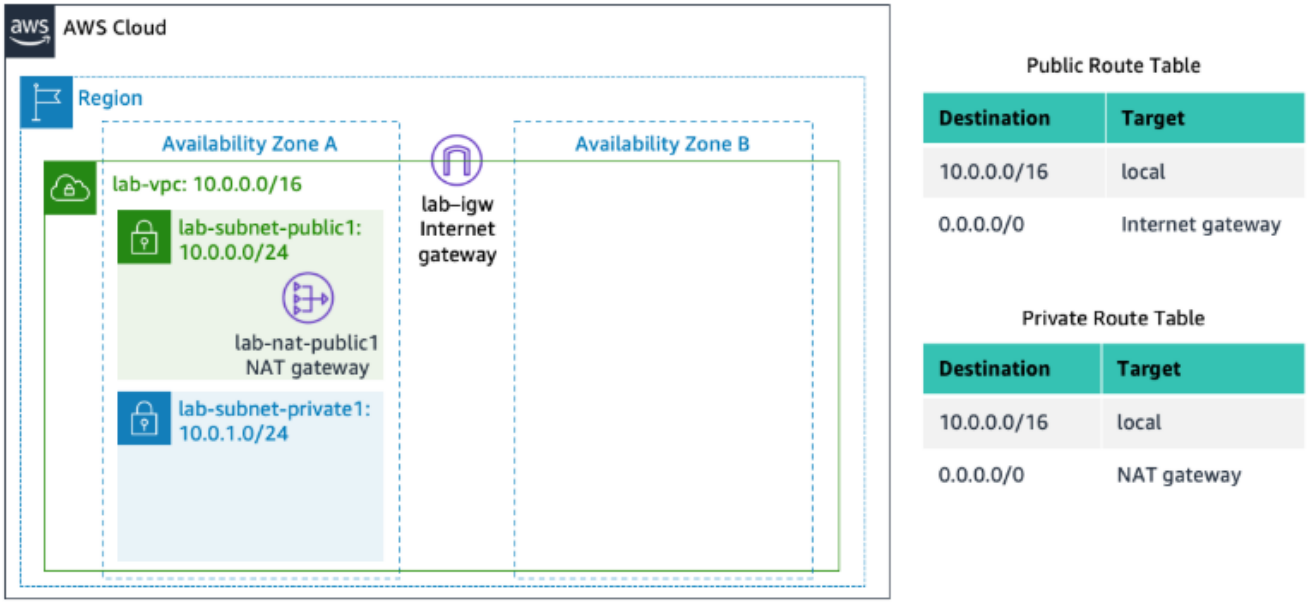


Figure 6. Original one-Availability-Zone architecture with the initial public and private subnets.

Subnet role	CIDR	Intended placement	Default internet route
Public1	10.0.0.0/24	First AZ	Internet Gateway
Private1	10.0.1.0/24	First AZ	NAT Gateway
Public2	10.0.2.0/24	Second AZ	Internet Gateway
Private2	10.0.3.0/24	Second AZ	NAT Gateway

The four subnet blocks are non-overlapping portions of the /16 VPC range. A subnet belongs to one Availability Zone, while the VPC can contain subnets in multiple zones. Adding two more subnets extends the network layout; it does not by itself make a single-server application highly available.

5. Creating the second public subnet and sizing IPv4

I created lab-subnet-public2 in the second Availability Zone with CIDR 10.0.2.0/24. The creation capture shows a success banner. The separate subnet-details image records 251 available IPv4 addresses for an empty 10.0.1.0/24 subnet, which led to the subnet-sizing note.

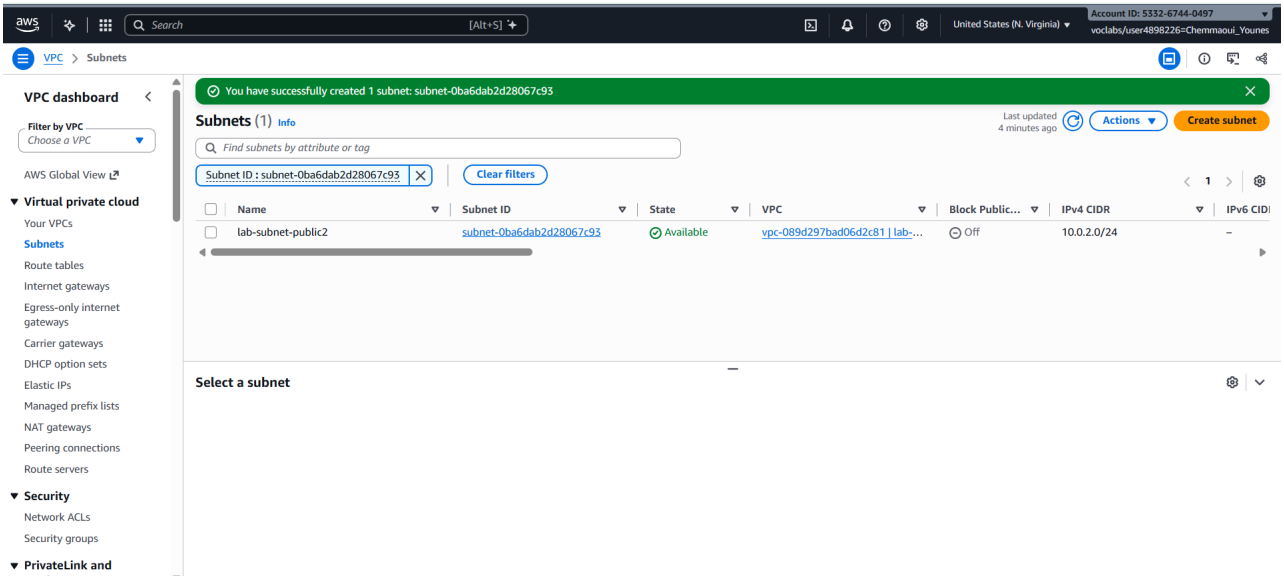


Figure 7. Original successful creation capture for lab-subnet-public2 with CIDR 10.0.2.0/24.

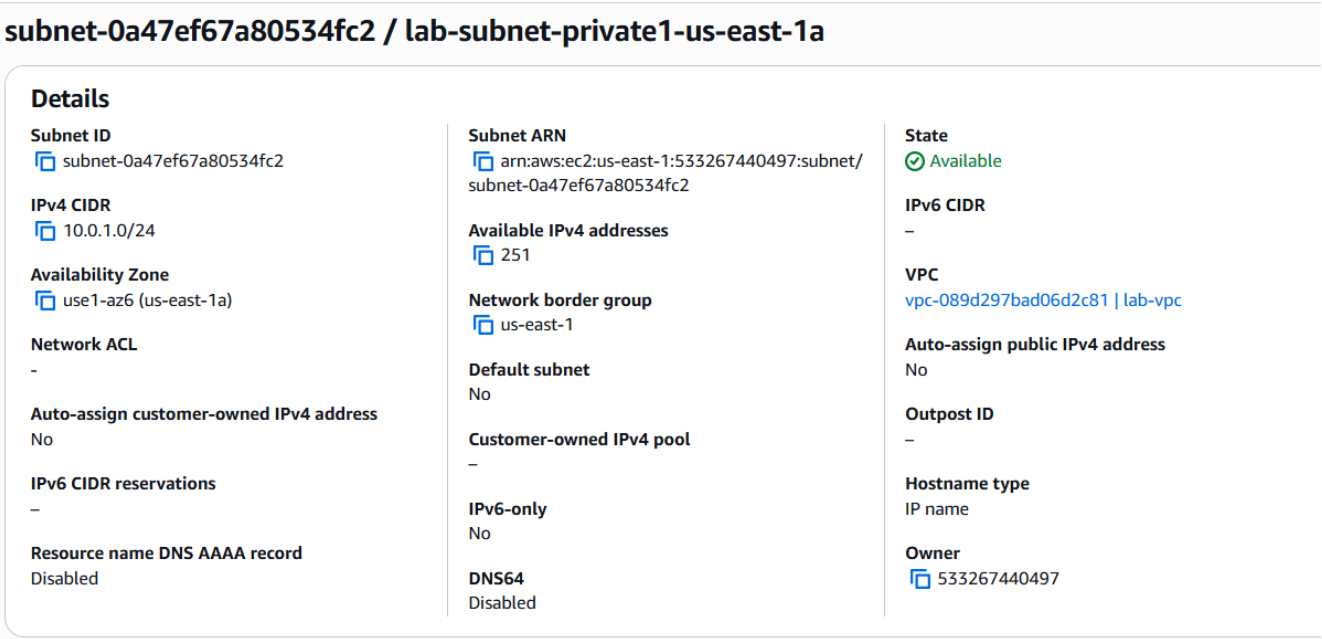


Figure 8. Subnet details showing a /24 block and 251 available IPv4 addresses.

A /24 contains 256 IPv4 addresses. In standard AWS VPC subnet addressing, five are reserved, leaving a nominal 251 before resource allocations. For 10.0.0.0/24 these are .0, .1, .2, .3 and .255. The linked note explains the network, router, reserved DNS-related position, future-use and broadcast reservations [1].

The VPC DNS resolver address is based on the primary VPC range plus two. Reserving the corresponding position in every subnet does not mean that each subnet has its own independent DNS server at that address. I did not configure an IPv6 lab in this exercise.

6. Creating the second private subnet

I then created lab-subnet-private2 in the second Availability Zone with CIDR 10.0.3.0/24. The resource map shows four subnets after expansion. The note next updates route-table associations so that the new subnets use the intended public or private path.

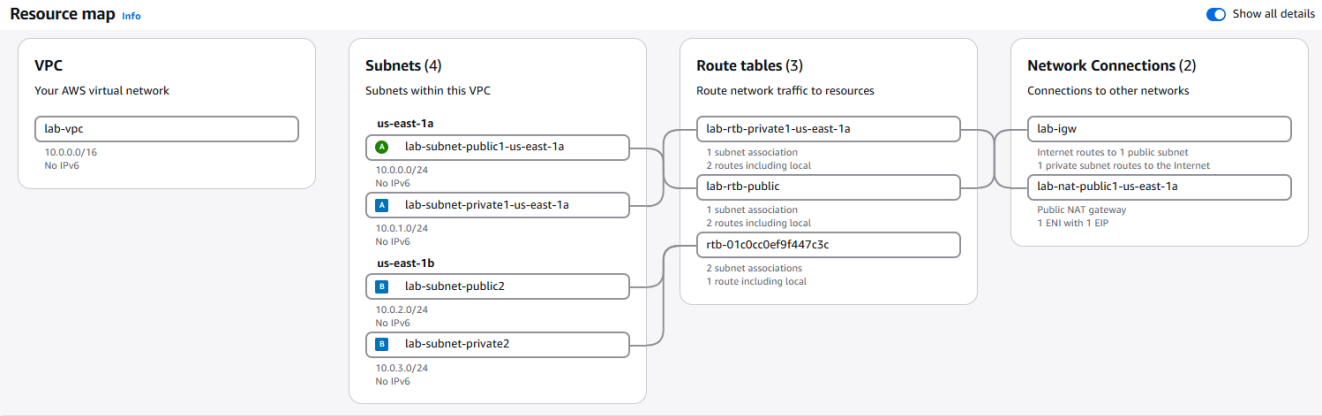


Figure 9. Original resource map with the four subnets visible after the second subnet pair was added.

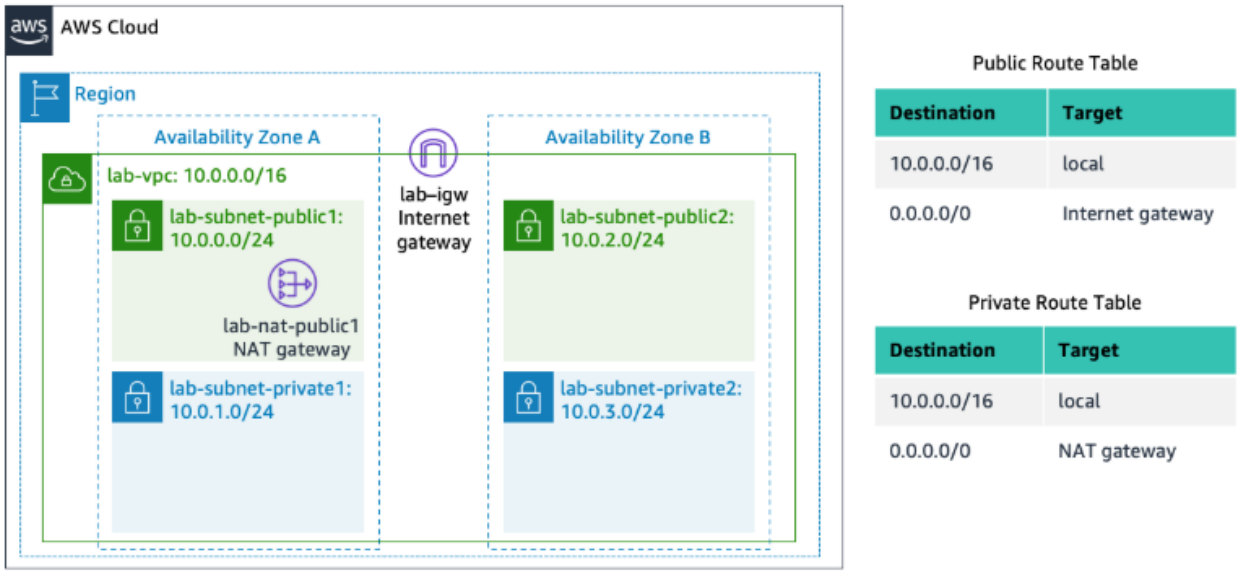


Figure 10. Original explanatory diagram of the expanded two-zone subnet layout before web-server placement.

Subnet creation and association are separate decisions. A newly created subnet can use the VPC main route table until an explicit association is set. I therefore inspected Routes and Subnet associations instead of assuming that a public/private name alone determines how the subnet routes traffic.

7. Private routing and subnet associations

The private route-table view shows 10.0.0.0/16 directed to local and 0.0.0.0/0 directed to a NAT Gateway. I used that route table for the private-subnet path. The raw and annotated route views are both retained.

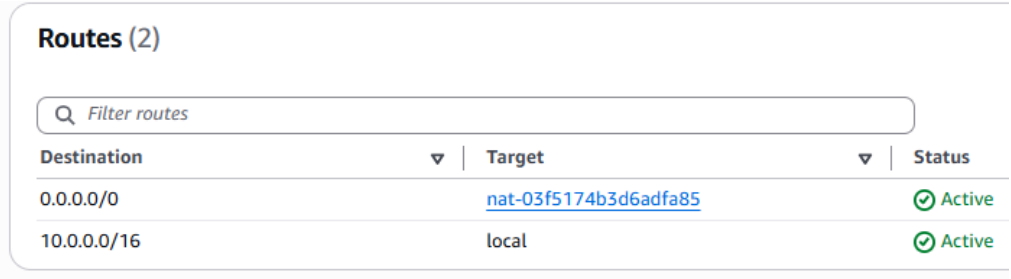


Figure 11. Original private route table with local routing and the default route to NAT.

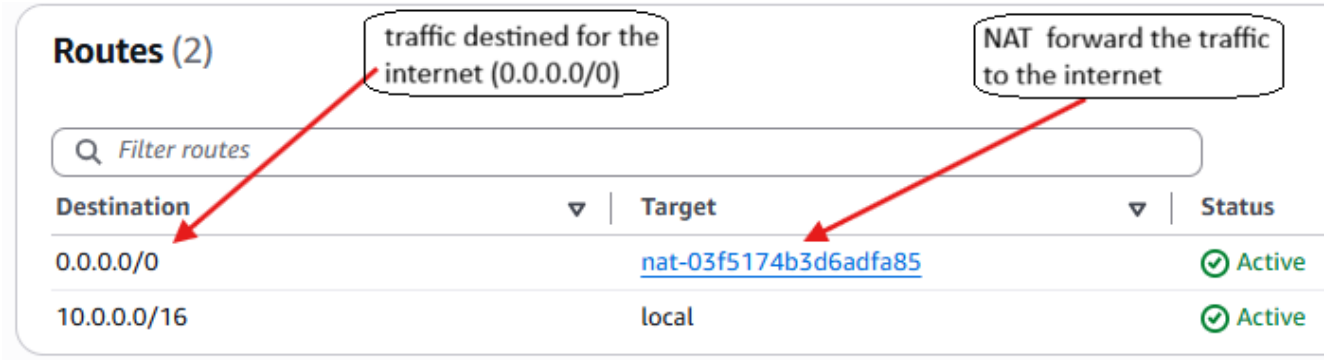


Figure 12. Original annotated NAT route view explaining the internet-bound destination.

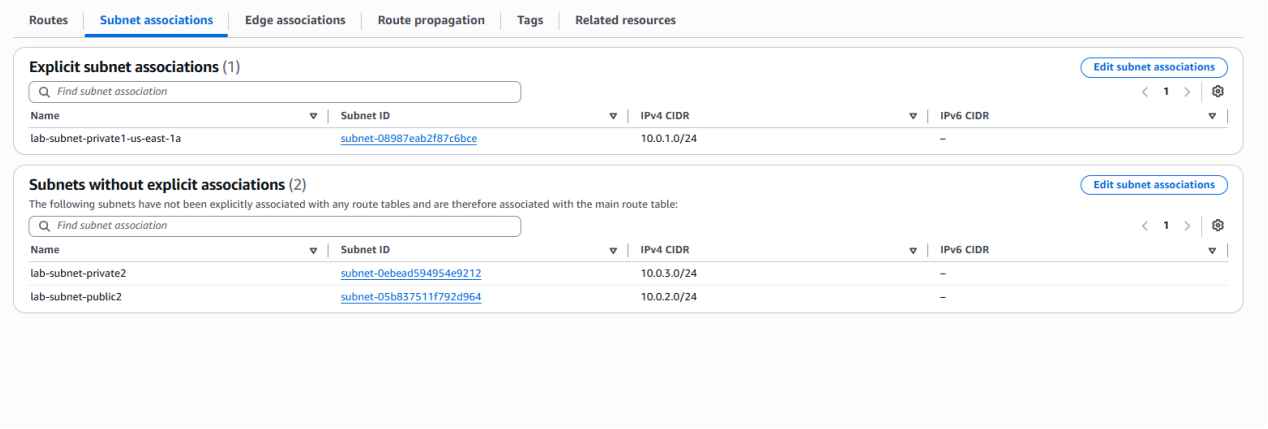


Figure 13. Original association view before adding private2: private1 is explicit and newly created subnets are listed without explicit associations.

The association screenshot is a before-state. The note records selecting both private subnets and saving the associations; the later grader awards the route-table association task 5/5. A public NAT Gateway supports connections initiated from private resources through the VPC internet path, rather than unsolicited inbound internet connections to those resources [2].

8. Public routing and the Internet Gateway

For lab-rtb-public, I verified the default IPv4 route to an Internet Gateway and recorded associating public2 alongside public1. The more-specific VPC-local route remains relevant for destinations inside 10.0.0.0/16; the default route is the fallback for other IPv4 destinations.

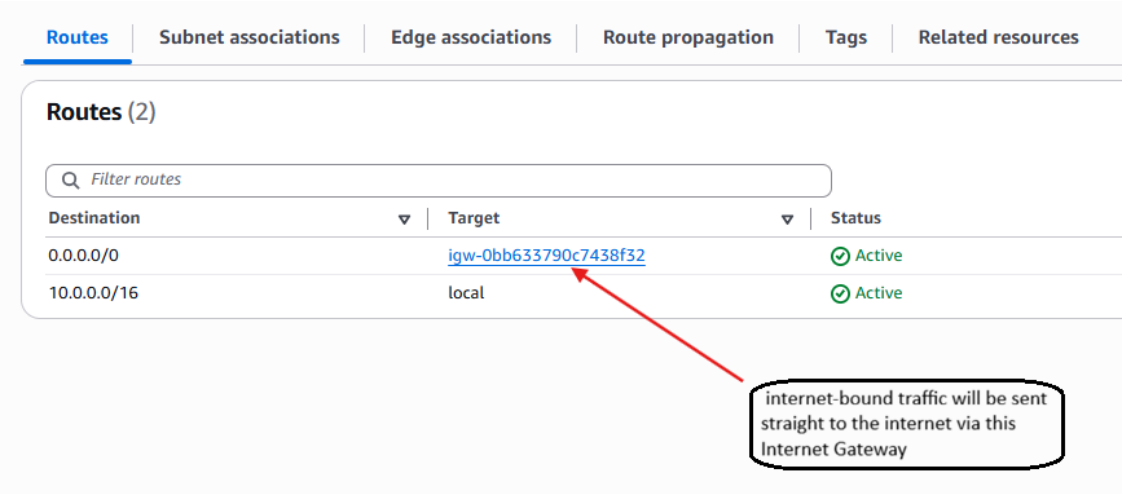


Figure 14. Original public route table with 0.0.0.0/0 directed to the Internet Gateway.

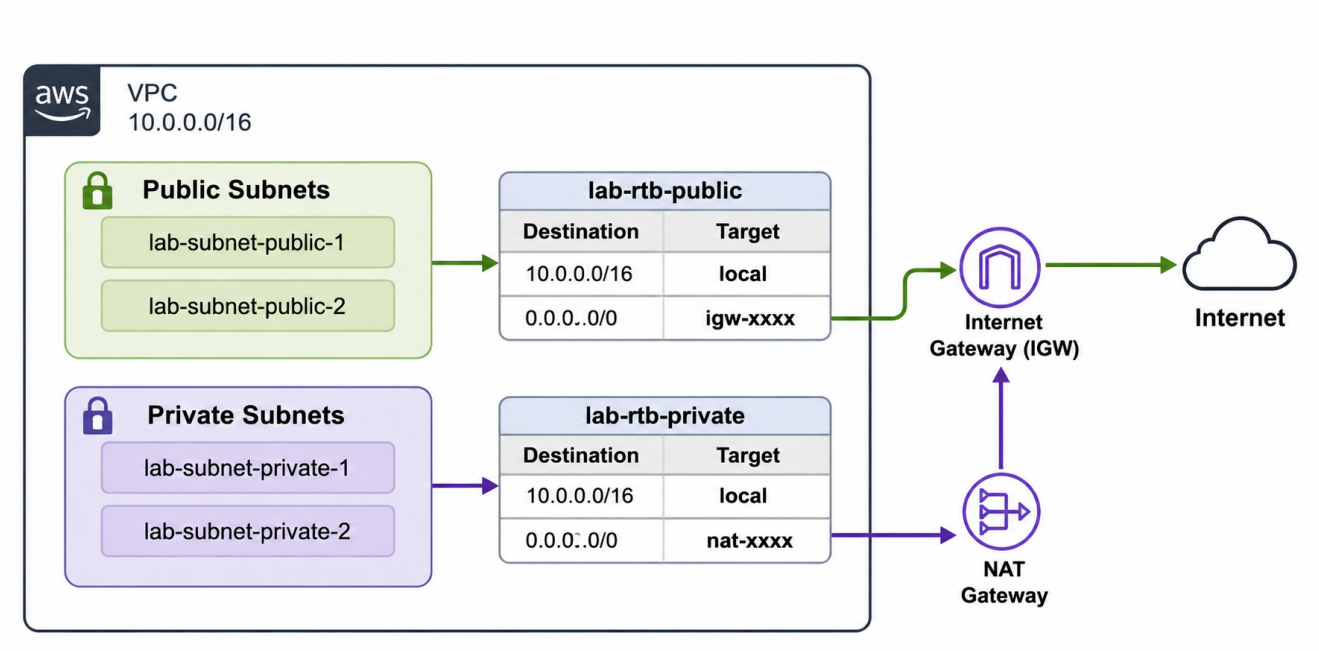


Figure 15. Original routing illustration contrasting the public Internet Gateway path and private NAT path.

A route table with an Internet Gateway path is what makes the subnet public in this design. An EC2 instance still needs a public IPv4 address for this IPv4 internet path, and security controls must permit the traffic [3]. The public and private labels alone do not grant access.

9. HTTP security group

I created Web Security Group in the lab VPC with description Enable HTTP access. The captured inbound rule allows HTTP, TCP port 80, from 0.0.0.0/0, with the description permit web requests. Both original versions of the success capture are retained.

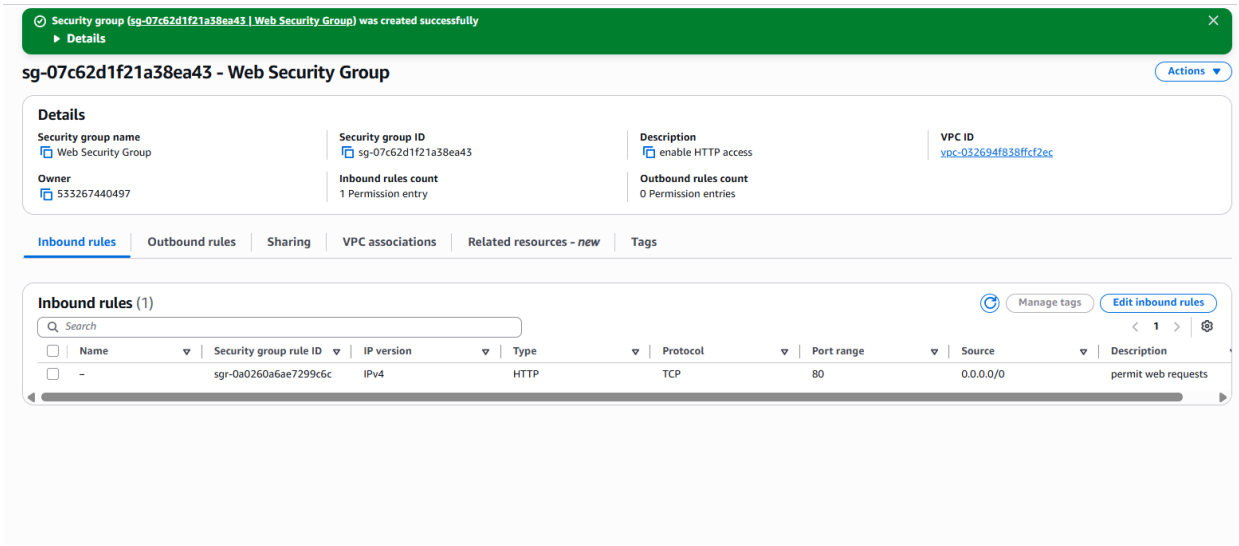


Figure 16. Original Web Security Group creation result and inbound HTTP rule.

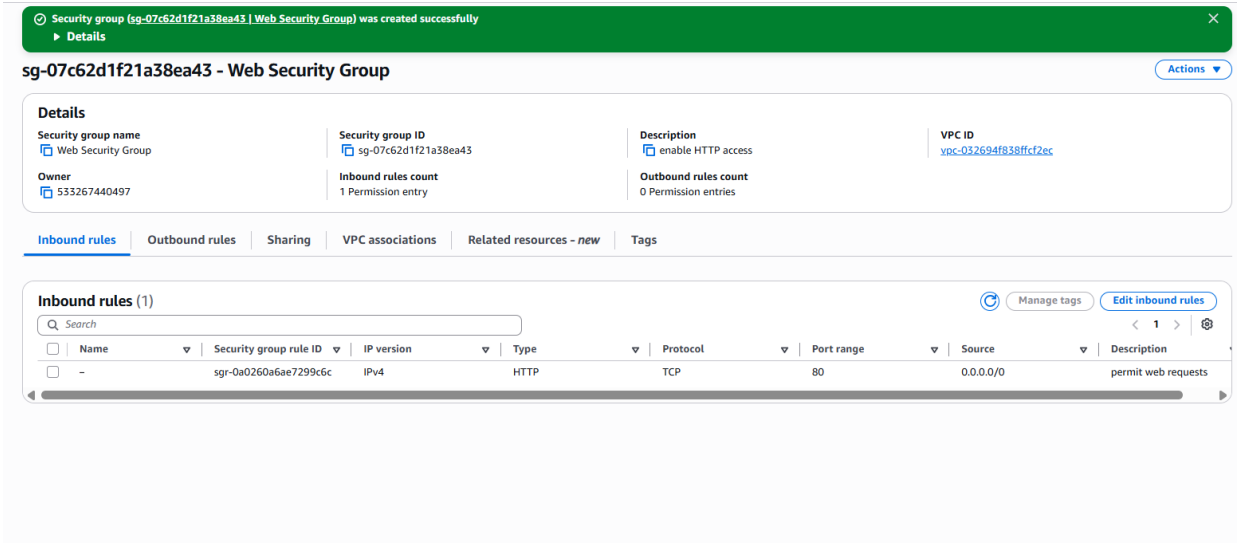


Figure 17. Second original security-group capture with the same inbound configuration.

The screenshot lists one inbound permission and zero outbound permission entries. I do not replace that observation with an assumed allow-all egress rule. Because later captures use another account/resource set, this image is not treated as a complete same-instance egress snapshot for the final browser test.

Security groups control traffic for associated resources [4]. HTTP access was the tested inbound path. Selecting an SSH key pair does not itself open SSH; no SSH login or port-22 access is claimed here. Outbound access needed for package downloads is an item to verify when reproducing the bootstrap workflow.

10. EC2 launch configuration

I launched Web Server 1 into the VPC using the existing Web Security Group. The working note specifies Amazon Linux 2023, t2.micro, key pair vockey, public2 as the subnet, and public IPv4 assignment enabled. The launch-summary capture is consistent with the small training instance and includes the user-data field.

Setting	Recorded value or purpose
Name	Web Server 1
AMI	Amazon Linux 2023
Instance type	t2.micro
Key pair	vockey, supplied by the training environment
Network placement	lab-vpc / lab-subnet-public2
Public address	Auto-assign public IPv4 enabled
Security group	Web Security Group, HTTP inbound
Storage	Default 8 GiB gp3 root storage described in the note

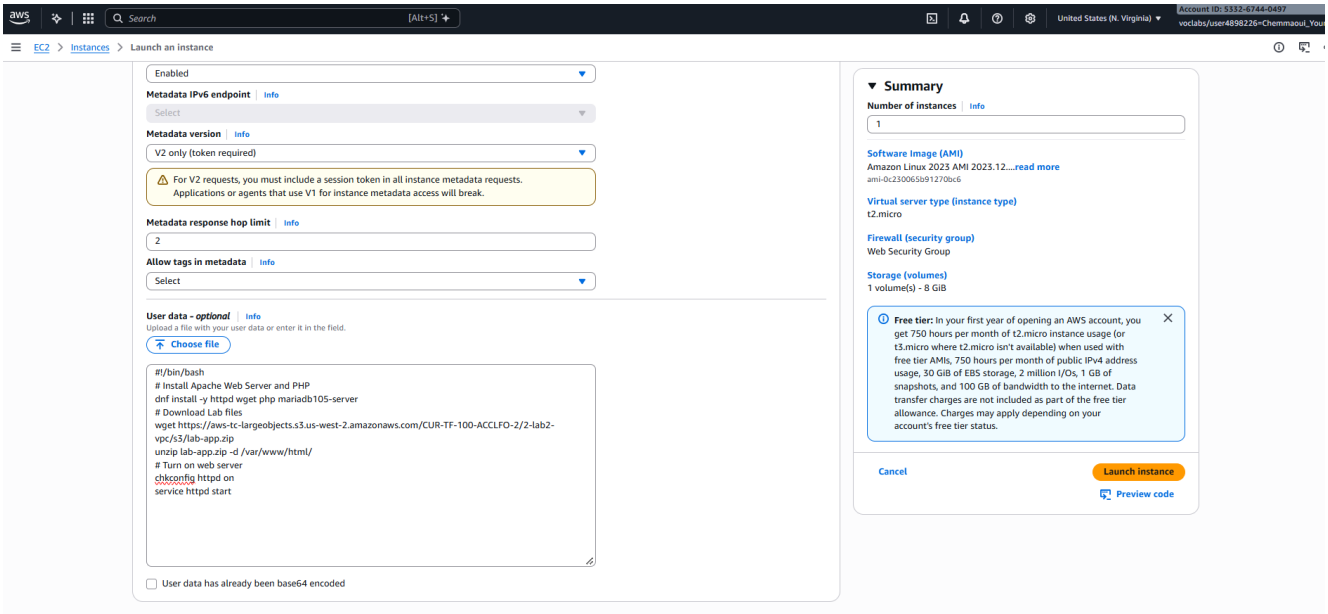


Figure 18. Original EC2 launch screen with the user-data script and launch summary.

The public subnet selection matters: the web server was intended to accept requests from outside the VPC. The private subnet path was not used for the public web endpoint. The key pair was selected as part of launch configuration, but no key material is included as a repository file.

11. User-data bootstrap and application installation

The working note records the following bootstrap script in the EC2 user-data field. It installs the web-service packages, downloads the supplied lab application, extracts it into the Apache document root and enables/starts httpd. This is a recorded script, not a command sequence newly executed for publication.

```
#!/bin/bash
# Install Apache Web Server and PHP
dnf install -y httpd wget php mariadb105-server
# Download Lab files
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-ACCLFO-2/2-lab2-vpc/s3/lab-app.zip
unzip lab-app.zip -d /var/www/html/
# Turn on web server
chkconfig httpd on
service httpd start
```

The dnf command requests the listed packages. wget retrieves the training archive, and unzip places the application under /var/www/html/. The last two lines express the intended persistent/startup and immediate service-start actions using the service-management commands supplied in the note.

EC2 user data is a mechanism for launch-time configuration [5]. The lab note describes root execution on the guest OS and first-launch behavior. However, the provided evidence is the configured field, the later instance state and the working page; a cloud-init log or exit status for each script line was not collected.

Installing mariadb105-server does not establish that a separate database service or RDS database was configured and tested. The application navigation contains an RDS label, but the documented infrastructure task is a VPC and EC2 web server. I do not infer a database deployment from a package name or navigation tab.

The public-instance route, outbound permissions, package availability, archive retrieval and service startup all matter when reproducing this workflow. The report preserves the recorded script rather than silently replacing commands with a different installation procedure.

12. Instance state and status checks

After launch, I waited for Web Server 1 to move from initializing status checks to the captured 2/2 checks passed state. Both the transitional and the later views are included so that a Running state is not mistaken for immediate application readiness.

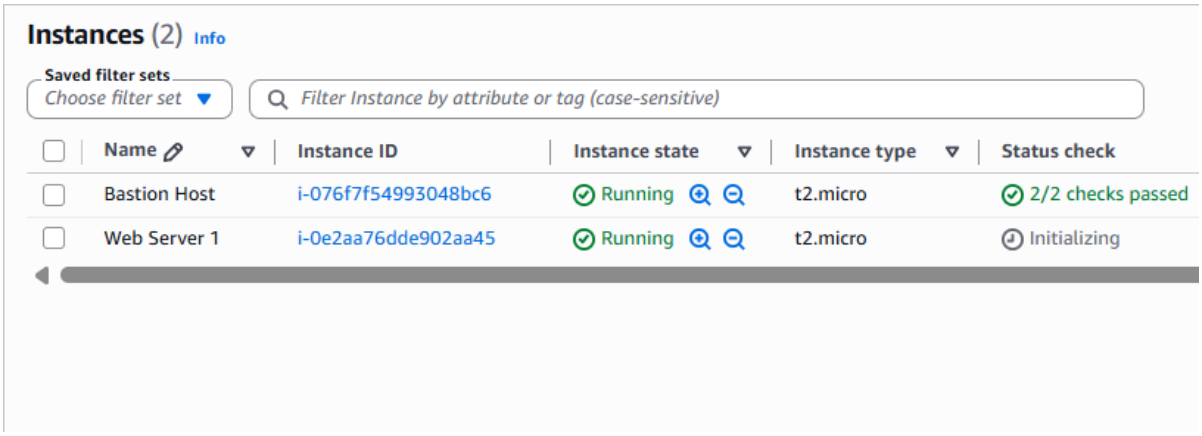


Figure 19. Original instance list: Web Server 1 is Running while checks are still initializing.

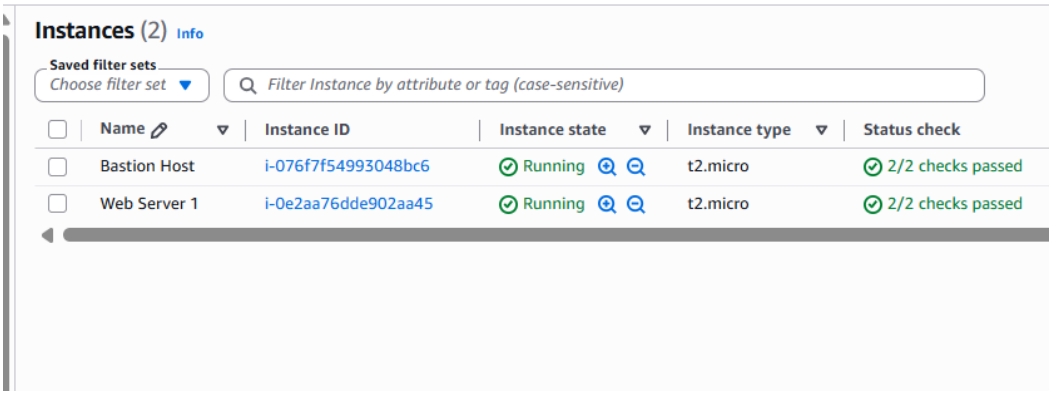


Figure 20. Later instance list: Web Server 1 shows 2/2 checks passed.

The same list contains a Bastion Host. The source does not record creating that resource as part of this workflow, so I treat it as account context rather than add it to the deployed web-server architecture.

Instance health checks and application reachability answer different questions. A healthy instance can still have an incorrect subnet association, blocked port, wrong protocol or stopped web service. I therefore continued to the public DNS and browser test instead of using the status-check result as the only success criterion.

13. Public DNS and network placement

I selected Web Server 1 and used the Public DNS value from its instance summary. The original and annotated captures show a running instance, a private address in 10.0.2.0/24, and a public IPv4/DNS entry. These details connect the browser test to the public2 placement described in the note.

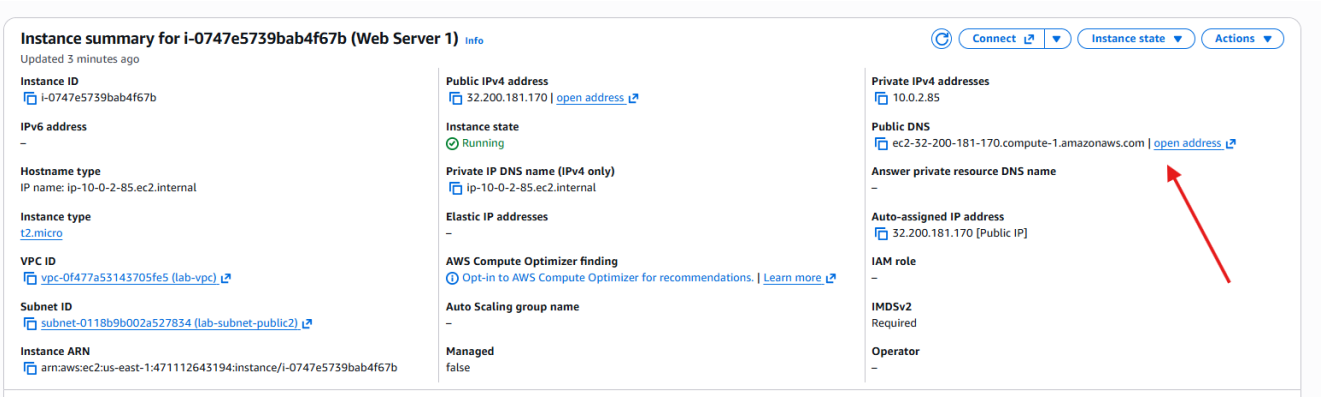


Figure 21. Original instance summary with the public address, public DNS and public2 subnet details.

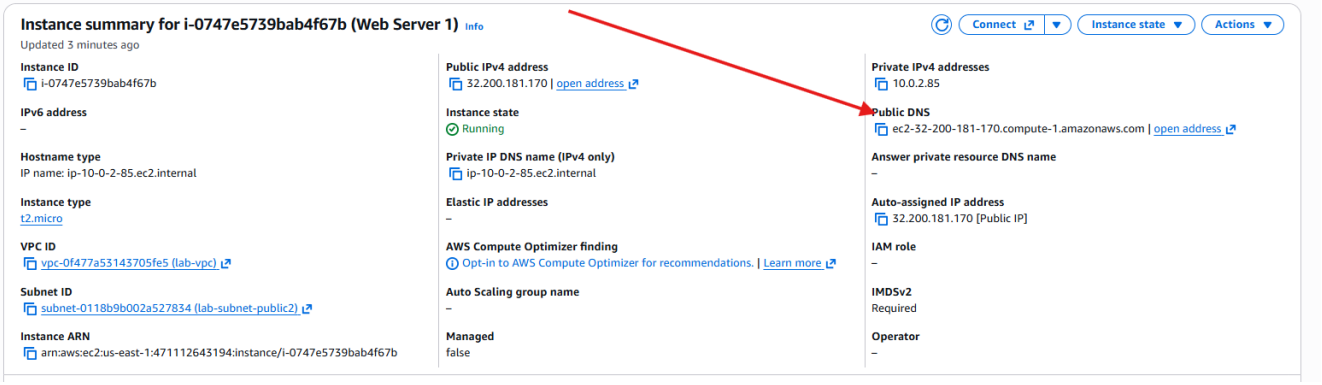


Figure 22. Annotated instance summary pointing to the Public DNS value used for the browser test.

The private address shown in the capture is part of the lab VPC subnet, while the public address/DNS is used from the external browser. A DNS name alone is not an access grant: the route, address assignment, security group and application listener still need to be correct.

14. Browser validation over HTTP

I opened the instance public DNS over HTTP. The source specifically notes using http rather than https. The captured page displays the AWS training application, the instance ID, Availability Zone and a current CPU-load value. This is direct evidence that the captured endpoint returned the application page.

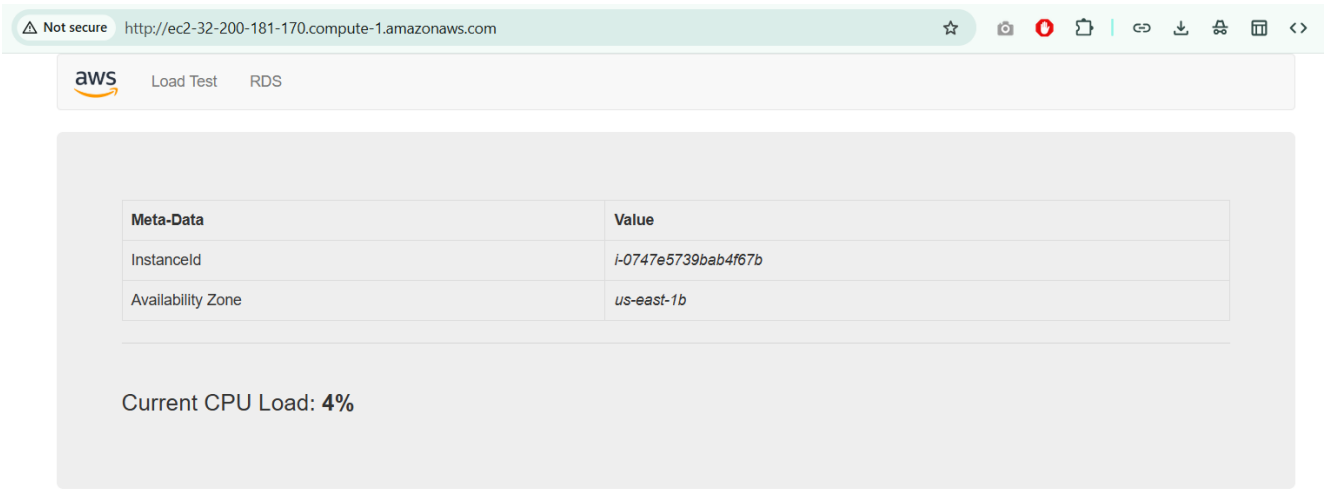


Figure 23. Original successful browser response from the EC2 web application over HTTP.

The result supports an end-to-end HTTP request path to that instance. It does not prove that every application feature works, that HTTPS is configured, or that the server remains reachable after the temporary training environment ends.

Check	What the source establishes
Instance state	Running with 2/2 status checks passed in the captured list
Address/placement	Public DNS and private 10.0.2.x address in public2
Inbound access	HTTP/TCP 80 rule captured
Application response	Browser displays the training application and instance metadata
Protocol scope	HTTP tested; TLS/HTTPS configuration not established

Allowing port 80 does not configure TLS, and choosing https would require the relevant listener, port and certificate setup. The browser result should therefore be described as a successful HTTP lab deployment rather than a production-secured website.

15. Grading result

The source includes a final grading summary of 30/30. The six listed tasks each receive 5/5. This is stronger outcome evidence than the instruction to submit the work or the course message congratulating the learner, because the score is visible in the captured result.

Total score	30/30
Task 1 - VPC created correctly	5/5
Task 2a - New subnets created correctly	5/5
Task 2b - Subnet route table association	5/5
Task 3 - Security group created correctly	5/5
Task 4a - EC2 instance created correctly	5/5
Task 4b - EC2 instance website accessible	5/5

Figure 24. Original grading summary showing 30/30 and six checks awarded 5/5.

The six checks cover VPC creation, new subnet creation, subnet route-table associations, security-group creation, EC2 instance creation and website accessibility. The report states the captured grade; it is not a claim that the grader was independently rerun during publication.

The source account/resource identifiers differ between early and late captures. The browser response and grading summary document a successful captured lab outcome, while the intermediate images explain the build and troubleshooting sequence across the saved sessions.

16. Technical discussion and lessons

The address plan and the traffic path should be considered together. Four /24 blocks fit inside the 10.0.0.0/16 VPC without overlapping. Their names express their intended roles, but the route table associated with each subnet determines its routing behavior.

The local route supports traffic within the VPC range. The default route covers destinations not matched by a more-specific route. In the public table that default points to the Internet Gateway; in the private table it points to NAT. This explains why adding a subnet requires checking its association rather than assuming that a CIDR or a public/private label is sufficient.

The public NAT Gateway and the Internet Gateway perform different roles in the private-subnet path. NAT enables private-resource connections outward without making those instances direct unsolicited inbound targets [2]. The publicly reachable server, by contrast, was placed in public2 with public IPv4 enabled and an HTTP ingress rule.

The two-Availability-Zone subnet layout is useful groundwork for resilient designs. This lab still has one documented web server and one NAT Gateway. It does not demonstrate application failover, load balancing, duplicated web tiers or resilient private egress under an Availability Zone failure.

The 251-address observation was another useful detail. A /24 is not 256 assignable EC2 addresses in a standard AWS subnet. Reserved positions reduce nominal capacity, and allocating resources further reduces the number shown as available. The linked CIDR note helped explain the Console value instead of treating it as an error.

Finally, I separated infrastructure health, application response and lab grading. The instance health result, the HTTP response and the 30/30 summary support different parts of the outcome. None should be used alone to assert a complete production readiness or security audit.

17. Limitations, reproducibility and references

The work is based on the saved Console captures and notes. No CLI transcript, infrastructure-as-code template, packet capture, NAT traffic test from a private instance, or guest bootstrap log was collected. I do not claim an SSH session, IPv6 deployment, HTTPS configuration, database validation or Availability Zone failure test.

The security-group screenshot reports zero outbound permission entries, while the later successful application capture belongs to another resource/account set. This makes a complete same-instance egress and bootstrap reconstruction an evidence gap. I preserve the original record instead of inventing an unseen rule change.

To make the next run more reproducible, I would record the final subnet associations and security-group inbound/outbound rules from the same deployment, retain user-data logs, and repeat the browser check with the same resource identifiers. Testing outbound access from a private instance would separately validate the NAT path.

The grading screenshot records 30/30, and the website capture records a working HTTP endpoint. The temporary public DNS shown in the images is a historical lab address, not a promise of a currently live public service.

All 24 local images, including repeated captures and diagrams, are included with unchanged bytes and original filenames. The main working note and the CIDR reference note remain in the Obsidian vault; raw course instructions are not redistributed as a full handout.

References

[1] Amazon VPC: Subnet CIDR blocks.
https://docs.aws.amazon.com/vpc/latest/userguide/subnet-sizing.html

[2] Amazon VPC: NAT gateways.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html

[3] Amazon VPC: Enable internet access using an internet gateway.
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html

[4] Amazon VPC: Control traffic using security groups.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html

[5] Amazon EC2: Run commands when you launch an instance with user data.
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html

Appendix. Complete original-image index

Every original local image is reproduced in this report and copied unchanged to assets/. Figure numbers locate its use in the narrative; filenames match the selected lab folder.

Figure	Original filename
15	AWS VPC Routing_ Public and Private Subnets.png
4	Pasted image 20261106195847.png
7	Pasted image 20261106222944.png
8	Pasted image 20261106223348.png
9	Pasted image 20261107004802.png
11	Pasted image 20261107005346.png
12	Pasted image 20261107010101.png
13	Pasted image 20261107010314.png
14	Pasted image 20261107010954.png
16	Pasted image 20261107012054.png
17	Pasted image 20261107012058.png
18	Pasted image 20261107012638.png
19	Pasted image 20261107012848.png
20	Pasted image 20261107013101.png
23	Pasted image 20261107023045.png
21	Pasted image 20261107023240.png
22	Pasted image 20261107023305.png
24	Pasted image 20261107023828.png
3	VPC-and-Web-Server-Lab-20261105172252.png
5	VPC-and-Web-Server-Lab-20261105180412.png
1	VPC-and-Web-Server-Lab-20261106151803.png
6	VPC-and-Web-Server-Lab-20261106151846.png
10	VPC-and-Web-Server-Lab-20261106151915.png
2	VPC-and-Web-Server-Lab-20261106151938.png